Aarya Arban
T11 – 05
Assignment No. 3

Aim: To implement Blinking LED and integrating sensors with Arduino

Theory:

Blinking LED using Arduino Uno:

A simple LED blinking circuit can be implemented using an external LED connected to an Arduino Uno. By toggling the digital pin HIGH and LOW with a time delay, the LED blinks continuously.

Ultrasonic Sensor Integration with Arduino:

The Ultrasonic Sensor (HC-SR04) measures distance by sending an ultrasonic pulse and measuring the time taken for it to bounce back. The Arduino reads this time and calculates the distance. The distance is displayed on the Serial Monitor.

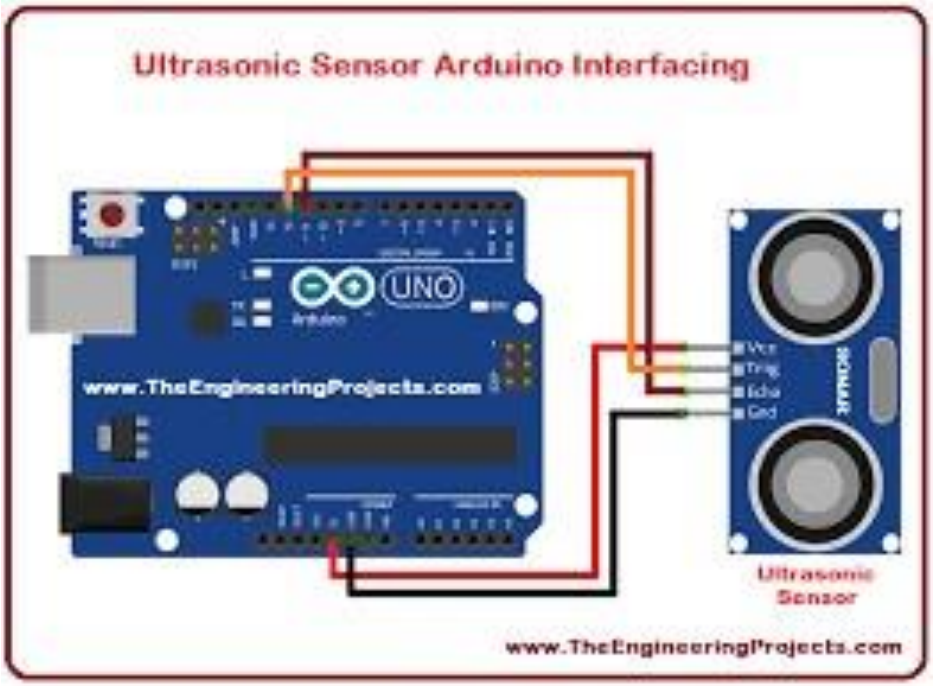
Condition-Based LED Activation:

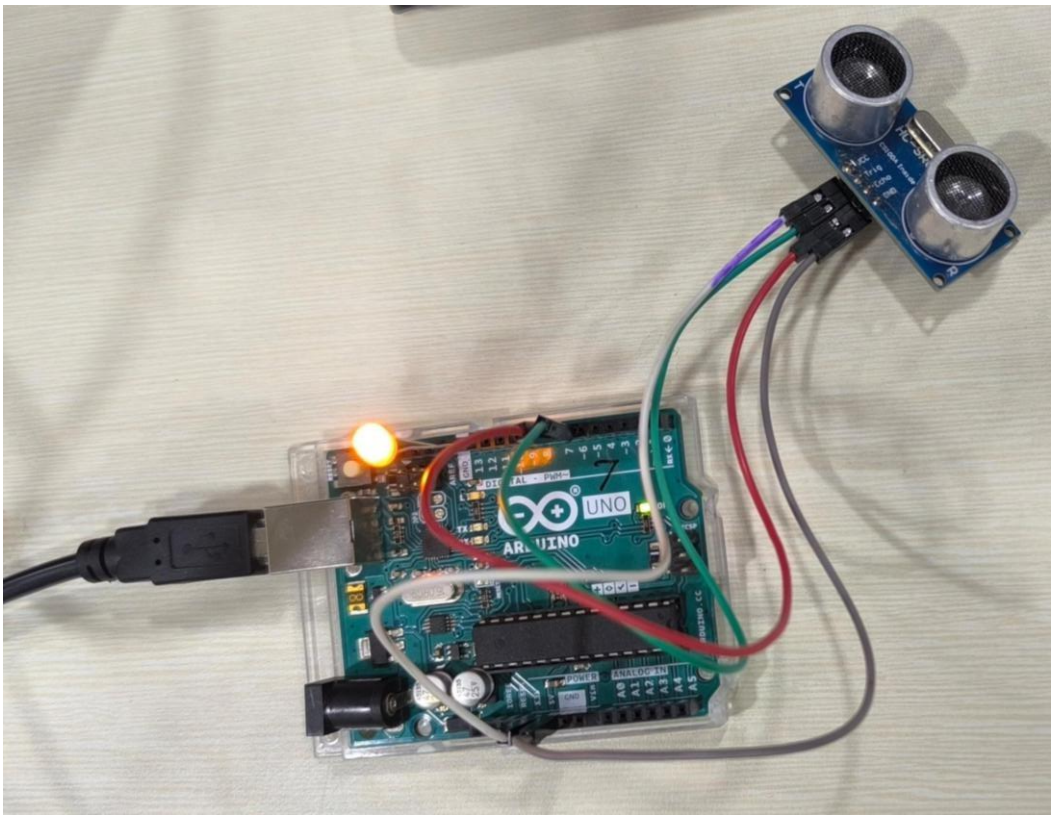
A condition can be added to glow the LED when the measured distance is below a certain threshold. For example, if an object is detected within 20 cm, the LED turns on; otherwise, it remains off.

How to Connect:

-VCC to 5V on the Arduino
-GND to GND on the Arduino
-Trig to digital pin 9 on the Arduino
-Echo to digital pin 10 on the Arduin

Diagram:


Ultrasonic Sensor Arduino Interfacing
UNO
Arduino
www.TheEngineeringProjects.com
Vcc
Trig
Echo
Gnd
Ultrasonic
Sensor
www.TheEngineeringProjects.com


UNO

Code:

sketch_jan28a.ino

```
const int trig = 6;          // Trigger pin
const int echo = 7;          // Echo pin
const int ledPin = 13;       // LED pin
long totaltime;              // Time for pulse to travel
int distance;                // Measured distance

void setup() {
  pinMode(trig, OUTPUT);     // Set the trig pin as an output
  pinMode(echo, INPUT);      // Set the echo pin as an input
  pinMode(ledPin, OUTPUT);   // Set the LED pin as an output
  Serial.begin(9600);        // Start serial communication
}

void loop() {
  digitalWrite(trig, LOW);           // Ensure trig pin is low
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);          // Send pulse
  delayMicroseconds(10);
  digitalWrite(trig, LOW);           // End pulse

  totaltime = pulseIn(echo, HIGH);   // Measure pulse time
  distance = totaltime * 0.034 / 2;  // Calculate distance in cm

  Serial.print("Distance: ");
  Serial.println(distance);          // Print distance to serial monitor

  if (distance > 20) {               // If the distance is more than 20 cm
    digitalWrite(ledPin, HIGH);      // Turn LED on
    delay(500);                      // Wait for 500 ms
    digitalWrite(ledPin, LOW);       // Turn LED off
    delay(500);                      // Wait for 500 ms
  }
  else {
    digitalWrite(ledPin, LOW);       // Turn LED off if distance is less than or equal to 20 cm
  }
}

```

Output:

```
15:49:07.897 -> Distance: 57 cm
15:49:08.427 -> Distance: 83 cm
15:49:08.913 -> Distance: 51 cm
15:49:09.408 -> Distance: 52 cm
15:49:09.912 -> Distance: 51 cm
15:49:10.432 -> Distance: 50 cm
15:49:10.924 -> Distance: 49 cm
15:49:11.467 -> Distance: 56 cm
15:49:11.936 -> Distance: 50 cm
15:49:12.455 -> Distance: 50 cm
15:49:12.955 -> Distance: 52 cm
15:49:13.488 -> Distance: 51 cm
15:49:13.954 -> Distance: 50 cm
15:49:14.507 -> Distance: 52 cm
15:49:14.993 -> Distance: 50 cm
15:49:15.474 -> Distance: 50 cm
15:49:16.008 -> Distance: 56 cm
15:49:16.513 -> Distance: 50 cm
15:49:17.074 -> Distance: 1206 cm
15:49:17.595 -> Distance: 37 cm
15:49:18.142 -> Distance: 1206 cm
15:49:18.677 -> Distance: 36 cm
15:49:19.174 -> Distance: 38 cm
```

Conclusion:

Understanding different sensors, controllers, and wireless modules helps in selecting the right components for IoT, automation, and robotics projects. Controllers like Arduino and Raspberry Pi provide flexible platforms for development, while communication modules like WiFi and GSM enable wireless connectivity, making them essential for smart applications.